

DocuMind Enterprise: Sistema Inteligente de Gestión y Análisis Documental

Fase 08: Matriz de trazabilidad integral de requisitos, código y Pruebas

Juan Esteban Rojas Meléndez

1. Propósito y Alcance de la Trazabilidad

El propósito de este documento es certificar la trazabilidad técnica bi-direccional y continua entre todas las fases del ciclo de ingeniería de software de DocuMind Enterprise. En este reporte en prosa y listas descriptivas, cada Requerimiento Funcional (RF), Requerimiento No Funcional (RNF) y Regla de Negocio (RN) definidos en la Fase 01 (Análisis) se asocia con su respectiva Historia de Usuario, Caso de Uso, componentes de código fuente en FastAPI o React (Fase 03) y el Caso de Prueba formal ejecutado para su validación (Fase 04).

2. Trazabilidad de Requerimientos Funcionales (RF)

- RF-01: Autenticación de Usuarios
 - Descripción: Inicio de sesión mediante credenciales cifradas y emisión de token seguro JSON Web Token (JWT).
 - Historia de Usuario: HU-01 (Autenticación Segura en DocuMind Enterprise).
 - Caso de Uso: CU-01 (Autenticación de Usuario).
 - Componentes de Código: Controladores de seguridad en backend/main.py y formulario de acceso en frontend/src/App.jsx.
 - Endpoint REST: POST /api/auth/login.
 - Caso de Prueba: CP-01.
 - Estado: Conforme.
- RF-02: Cierre de Sesión
 - Descripción: Finalización e invalidación de la sesión activa con purga del token local en el cliente.
 - Historia de Usuario: HU-01 (Autenticación Segura en DocuMind Enterprise).
 - Caso de Uso: CU-01 (Autenticación de Usuario).
 - Componentes de Código: Limpieza de estado y almacenamiento en frontend/src/App.jsx y botón en la barra lateral.
 - Mecanismo: Purga del almacenamiento local del navegador (localStorage.removeItem).
 - Caso de Prueba: CP-01.
 - Estado: Conforme.

- RF-03: Protección de Rutas
 - Descripción: Denegación de acceso a operaciones del repositorio y vistas a peticiones no autenticadas.
 - Historia de Usuario: HU-01 (Autenticación Segura en DocuMind Enterprise).
 - Caso de Uso: CU-01 (Autenticación de Usuario).
 - Componentes de Código: Dependencia HTTPBearer en backend/main.py y rutas protegidas condicionales en frontend/src/App.jsx.
 - Mecanismo: Encabezado HTTP Authorization: Bearer <token>.
 - Caso de Prueba: CP-01.
 - Estado: Conforme.
- RF-04: Administración de Carpetas y Repositorios
 - Descripción: Creación, listado lógico y organización modular de los archivos corporativos.
 - Historia de Usuario: HU-02 (Carga y Validación Documental).
 - Caso de Uso: CU-02 (Cargar y Procesar Documento).
 - Componentes de Código: Modelos de persistencia en backend/app/models.py y módulo de repositorio en frontend/src/App.jsx.
 - Endpoints REST: GET /api/documents y gestión lógica de colecciones.
 - Caso de Prueba: CP-03.
 - Estado: Conforme.
- RF-05: Carga Multiformato
 - Descripción: Admisión estricta y almacenamiento físico de archivos en extensiones exclusivas PDF, DOCX y TXT.
 - Historia de Usuario: HU-02 (Carga y Validación Documental).
 - Caso de Uso: CU-02 (Cargar y Procesar Documento).
 - Componentes de Código: Subida asíncrona en backend/main.py y componente interactivo en frontend/src/App.jsx.
 - Endpoint REST: POST /api/documents/upload.
 - Casos de Prueba: CP-02 y CP-03.
 - Estado: Conforme.
- RF-06: Consulta y Descarga de Archivos
 - Descripción: Listado del acervo documental y descarga directa del archivo fuente original en su formato binario.
 - Historia de Usuario: HU-02 (Carga y Validación Documental).

- Caso de Uso: CU-02 (Cargar y Procesar Documento).
- Componentes de Código: Rutas de entrega de archivos en backend/main.py y panel de detalle en el frontend.
- Endpoints REST: GET /api/documents/{id} y GET /api/documents/{id}/download.
- Caso de Prueba: CP-03.
- Estado: Conforme.
- RF-07: Eliminación Sincronizada
 - Descripción: Supresión coordinada del archivo físico en disco junto con sus metadatos y fragmentos vectoriales en base de datos.
 - Historia de Usuario: HU-02 (Carga y Validación Documental).
 - Caso de Uso: CU-04 (Eliminar Archivo).
 - Componentes de Código: Rutas de eliminación en backend/main.py y restricción de clave foránea en backend/app/models.py.
 - Endpoint REST: DELETE /api/documents/{id}.
 - Caso de Prueba: CP-09.
 - Estado: Conforme.
- RF-08: Extracción de Texto Plano
 - Descripción: Extracción del contenido textual digital según el tipo MIME del archivo (pdfplumber, python-docx o lectura UTF-8).
 - Historia de Usuario: HU-03 (Procesamiento Automatizado con Inteligencia Artificial).
 - Caso de Uso: CU-02 (Cargar y Procesar Documento).
 - Componentes de Código: Funciones de análisis de texto en backend/main.py.
 - Mecanismo: Procesamiento interno durante la recepción en /api/documents/upload.
 - Caso de Prueba: CP-03.
 - Estado: Conforme.
- RF-09: Clasificación Automática con IA
 - Descripción: Catalogación autónoma del expediente dentro de Facturas, Contratos u Hojas de Vida mediante Google Gemini.
 - Historia de Usuario: HU-03 (Procesamiento Automatizado con Inteligencia Artificial).
 - Caso de Uso: CU-02 (Cargar y Procesar Documento).
 - Componentes de Código: Integración con gemini-3.6-flash en backend/main.py y persistencia en backend/app/models.py.
 - Mecanismo: System Prompt con validación de categorías.
 - Casos de Prueba: CP-04, CP-05 y CP-06.

- Estado: Conforme.
- RF-10: Resumen Automático
 - Descripción: Síntesis ejecutiva de 2 a 4 oraciones sobre el contenido del documento procesado.
 - Historia de Usuario: HU-03 (Procesamiento Automatizado con Inteligencia Artificial).
 - Caso de Uso: CU-02 (Cargar y Procesar Documento).
 - Componentes de Código: Generación de resúmenes en backend/main.py y despliegue visual en frontend/src/App.jsx.
 - Mecanismo: Columna de texto summary en la base de datos.
 - Casos de Prueba: CP-04, CP-05 y CP-06.
 - Estado: Conforme.
- RF-11: Extracción Estructurada para Facturas
 - Descripción: Identificación y almacenamiento en esquema JSON de número de factura, emisor, cliente, fecha y total.
 - Historia de Usuario: HU-04 (Extracción Estructurada de Información Clave).
 - Caso de Uso: CU-02 (Cargar y Procesar Documento).
 - Componentes de Código: Esquema estructurado en backend/main.py y renderizado en modal de detalle.
 - Mecanismo: Columna extracted_data tipo JSONB en PostgreSQL.
 - Caso de Prueba: CP-04.
 - Estado: Conforme.
- RF-12: Extracción Estructurada para Contratos
 - Descripción: Identificación y almacenamiento en esquema JSON de tipología, partes involucradas, objeto, vigencia y valor pactado.
 - Historia de Usuario: HU-04 (Extracción Estructurada de Información Clave).
 - Caso de Uso: CU-02 (Cargar y Procesar Documento).
 - Componentes de Código: Esquema estructurado en backend/main.py y renderizado en modal de detalle.
 - Mecanismo: Columna extracted_data tipo JSONB en PostgreSQL.
 - Caso de Prueba: CP-05.
 - Estado: Conforme.
- RF-13: Extracción Estructurada para Hojas de Vida
 - Descripción: Identificación y almacenamiento en esquema JSON de candidato, rol principal, años de experiencia y competencias técnicas.

- Historia de Usuario: HU-04 (Extracción Estructurada de Información Clave).
- Caso de Uso: CU-02 (Cargar y Procesar Documento).
- Componentes de Código: Esquema estructurado en backend/main.py y renderizado en modal de detalle.
- Mecanismo: Columna extracted_data tipo JSONB en PostgreSQL.
- Caso de Prueba: CP-06.
- Estado: Conforme.
- RF-14: Vectorización e Indexación para RAG
 - Descripción: Segmentación de texto en fragmentos (chunks), cálculo de vectores de 768 dimensiones e inserción en pgvector.
 - Historia de Usuario: HU-05 (Consulta en Lenguaje Natural).
 - Caso de Uso: CU-02 (Cargar y Procesar Documento).
 - Componentes de Código: Invocación a gemini-embedding-2 en backend/main.py y modelo DocumentChunk en backend/app/models.py.
 - Mecanismo: Columna embedding Vector(768) en PostgreSQL.
 - Caso de Prueba: CP-07.
 - Estado: Conforme.
- RF-15: Búsqueda Documental por Metadatos
 - Descripción: Filtros reactivos para localizar expedientes por nombre, tipología asignada o estado.
 - Historia de Usuario: HU-05 (Consulta en Lenguaje Natural).
 - Caso de Uso: CU-03 (Consulta Documental mediante RAG).
 - Componentes de Código: Consulta parametrizada en backend/main.py y barra de filtrado dinámico en frontend/src/App.jsx.
 - Endpoint REST: GET /api/documents?search=...&category=...
 - Caso de Prueba: CP-03.
 - Estado: Conforme.
- RF-16: Consulta en Lenguaje Natural (Motor RAG)
 - Descripción: Resolución de preguntas formuladas en lenguaje natural recuperando fragmentos por similitud de coseno y citando fuentes.
 - Historia de Usuario: HU-05 (Consulta en Lenguaje Natural).
 - Caso de Uso: CU-03 (Consulta Documental mediante RAG).
 - Componentes de Código: Cálculo de similitud coseno en backend/main.py e interfaz conversacional en frontend/src/App.jsx.
 - Endpoint REST: POST /api/rag/query.
 - Casos de Prueba: CP-07 y CP-08.

- Estado: Conforme.
- RF-17: Dashboard de Indicadores
 - Descripción: Panel central de métricas con conteo total de documentos, distribución categórica y estados de procesamiento.
 - Historia de Usuario: HU-06 (Monitoreo Global y Registro de Fallos).
 - Caso de Uso: CU-01 (Autenticación de Usuario).
 - Componentes de Código: Agregaciones SQL en backend/main.py y tarjetas de indicadores en frontend/src/App.jsx.
 - Endpoint REST: GET /api/dashboard/stats.
 - Caso de Prueba: CP-03.
 - Estado: Conforme.
- RF-18: Registro y Trazabilidad de Fallos
 - Descripción: Gestión de ciclo de estados (Pendiente, Procesando, Procesado, Error) y registro de la traza de fallos en error_log.
 - Historia de Usuario: HU-06 (Monitoreo Global y Registro de Fallos).
 - Caso de Uso: CU-02 (Cargar y Procesar Documento).
 - Componentes de Código: Bloques de captura try-except en backend/main.py y campo error_log en backend/app/models.py.
 - Mecanismo: Columna de texto error_log para auditoría técnica.
 - Caso de Prueba: CP-10.
 - Estado: Conforme.

3. Trazabilidad de Requerimientos No Funcionales (RNF)

- RNF-01: Seguridad de Credenciales y Variables de Entorno
 - Criterio Técnico: Secretos, API keys de Google Gemini y llaves JWT aislados de los repositorios de código.
 - Implementación: Archivo backend/.env excluido formalmente del control de versiones mediante el archivo .gitignore.
 - Verificación: Inspección de commits en Git y carga de parámetros con pydantic-settings / os.getenv.
- RNF-02: Rendimiento en Ingesta Documental
 - Criterio Técnico: Tiempo de extracción de texto y consulta con el LLM inferior a 15 segundos en documentos de hasta 10 páginas.
 - Implementación: Pipeline asíncrono en FastAPI con el modelo optimizado gemini-3.6-flash.
 - Verificación: Tiempos de ingesta registrados entre 2.8 y 4.5 segundos en las pruebas del corpus de 30 documentos.

- RNF-03: Tiempo de Respuesta en Motor RAG
 - Criterio Técnico: Cálculo de embedding de la consulta, búsqueda vectorial y generación de respuesta en menos de 5 segundos.
 - Implementación: Búsqueda por similitud de coseno en pgvector acotada a los 3 fragmentos más cercanos.
 - Verificación: Consultas respondidas con una latencia promedio de 2.1 segundos en el endpoint /api/rag/query.
- RNF-04: Validación Estricta de Formatos de Archivo
 - Criterio Técnico: Validación de extensiones permitidas y verificación de cabeceras de archivo para rechazo inmediato de formatos inválidos.
 - Implementación: Filtro de entrada en FastAPI validando extensiones .pdf, .docx y .txt junto con validación MIME.
 - Verificación: Caso de prueba CP-02 con rechazo de archivos binarios ejecutables o imágenes sueltas.
- RNF-05: Indexación Vectorial Eficiente
 - Criterio Técnico: Persistencia vectorial con soporte para índices de aproximación de vecinos cercanos en pgvector.
 - Implementación: Índice HNSW (idx_document_chunks_embedding_hnsw) configurado sobre la tabla document_chunks.
 - Verificación: Plan de consulta SQL verificado con EXPLAIN ANALYZE en PostgreSQL.
- RNF-06: Usabilidad y Diseño Adaptable
 - Criterio Técnico: Interfaz reactiva con Tailwind CSS que ofrece feedback de estados semánticos y optimización de pantalla completa.
 - Implementación: Aplicación SPA en React con badges de estado por colores, modales dinámicos e indicadores de carga.
 - Verificación: Pruebas de usabilidad en resoluciones de 1366x768 y 1920x1080 píxeles.
- RNF-07: Reproducibilidad de Despliegue
 - Criterio Técnico: Configuración integral para permitir la ejecución local o en servidor sin conflictos de dependencias del sistema operativo.
 - Implementación: Contenedor pgvector/pgvector:pg16 vía Docker Compose, requirements.txt y scripts de inicialización DDL.

- Verificación: Despliegue limpio comprobado en el manual técnico desde una instalación limpia de Docker.

4. Trazabilidad de Reglas de Negocio (RN)

- RN-01: Obligatoriedad de Autenticación
 - Regla: Ninguna operación de lectura, subida, eliminación o consulta RAG es accesible sin una sesión válida activa.
 - Implementación: Middleware de seguridad e inyección de dependencias Depends(get_current_user) en FastAPI.
- RN-02: Restricción de Formatos y Tamaño
 - Regla: Se admiten únicamente archivos .pdf, .docx y .txt con un límite estricto de hasta 10 MB.
 - Implementación: Validación en el backend de la longitud de contenido (Content-Length) y de la extensión del archivo.
- RN-03: Clasificación Categórica Estricta
 - Regla: Todo archivo analizado por IA debe asignarse exclusivamente a Facturas, Contratos u Hojas de Vida.
 - Implementación: Restricción en el System Prompt de Gemini condicionando la salida a una de las tres tipologías requeridas.
- RN-04: Esquema de Extracción Según Tipología
 - Regla: El motor de IA aplica un esquema JSON adaptado a la categoría identificada.
 - Implementación: Generación condicionada de campos requeridos (emisor/total para facturas, partes/vigencia para contratos, experiencia/habilidades para perfiles).
- RN-05: Secuencia Obligatoria de Transición de Estados
 - Regla: El ciclo de vida transiciona por Pendiente $\rightarrow$ Procesando $\rightarrow$ Procesado; ante fallos de extracción o API conmuta a Error.
 - Implementación: Actualizaciones atómicas en base de datos al ingresar a la canalización y captura de excepciones con persistencia en error_log.
- RN-06: Integridad Referencial en Eliminación
 - Regla: Al eliminar un documento se suprimen en cascada sus metadatos, resúmenes y fragmentos vectoriales.

- Implementación: Restricción relacional ON DELETE CASCADE en la clave foránea de document_chunks y eliminación del archivo en disco.
- RN-07: Protección de Datos Personales
 - Regla: El corpus de 30 documentos de prueba no debe contener datos personales ni financieros reales sin consentimiento.
 - Implementación: Generación del acervo de prueba mediante el script sintético generate_mock_data.py.